

Software Requirements Specification

SmartRetail

Electronics POS & Installment Management Platform

1. Introduction

1.1 Purpose

This document specifies the complete software requirements for SmartRetail, a retail management platform for electronics businesses handling full-payment and installment sales. It describes the system's external behavior at a functional level of detail sufficient for two developers to design against and for a reviewer or tester to verify against, along with usability, reliability, performance, security, supportability, and interface requirements that apply across the whole system.

The document deliberately separates every requirement into either the core system, which must be fully working and demonstrable, or an optional extension, which is attempted only once the core is complete. This separation is a project-management decision made necessary by the two-person team size and short delivery window, and it is repeated throughout Section 3 wherever it applies.

1.2 Scope

SmartRetail covers point-of-sale checkout, product and inventory management, customer records, installment sales with automated reminders, invoice and report generation, and — as an optional extension — basic decision-support analytics (demand forecasting, installment risk scoring, reorder suggestions, and anomaly flags).

In scope	Out of scope for this delivery
POS checkout, inventory, customers, installments, invoices, reports, RBAC, audit logging, CI/CD, Docker deployment	Supplier / procurement workflow, warranty claim processing as a distinct module, refunds as a distinct reversal engine
Temporal-based installment reminder and overdue workflow	General-purpose workflow builder beyond installments
Two-role access control (Administrator, Worker)	Multi-branch / multi-tenant retail chains
Optional: demand forecasting, risk scoring, reorder suggestion, anomaly flags	LLM-based agents, Kafka, Kubernetes, microservices, data lakehouse, deep learning

Anything not listed in the left-hand column above is explicitly out of scope for this delivery and is not covered by the requirements in Section 3.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
POS	Point of Sale — the checkout process and screen used to record a sale.
RBAC	Role-Based Access Control — restricting system actions based on a user's assigned role.
Installment plan	An agreement to pay for a purchase in scheduled parts instead of in full at checkout.
Deposit	The initial payment made at checkout on an installment plan, before the remaining balance is financed.
Temporal	A workflow orchestration engine used to run long-running, durable background processes such as installment reminders.
Idempotency key	A unique value attached to a request so that resubmitting it does not create a duplicate transaction.
JWT	JSON Web Token — a signed token used to authenticate a logged-in user's requests.
Stock movement	A single recorded change to a product's stock quantity (purchase, sale, return, damage, or adjustment).
Serialized unit	An individual product instance tracked by its own serial number or IMEI, typically for warranty purposes.
SKU	Stock Keeping Unit — a unique identifier for a distinct product.
MTBF / MTTR	Mean Time Between Failures / Mean Time To Repair — reliability measures defined in Section 3.3.
Background job	A task (e.g., report generation) that runs outside the request/response cycle so the user interface is not blocked while it completes.
Core requirement	A requirement (FR1–FR8 and associated non-functional requirements) that must be fully implemented for the delivery to be considered complete.
Optional requirement	A requirement (FR9 and its sub-features) attempted only after all core requirements are implemented and tested.

1.4 References

See Section 4, Supporting Information, for the full reference list.

1.5 Overview

Section 2 describes the system at a general level, including its perspective, functions, users, constraints, and assumptions. Section 3 lists the specific functional and non-functional requirements in detail, including every core and optional feature. Section 4 provides supporting references and appendices.

2. Overall Description

2.1 Product Perspective

SmartRetail is a new, self-contained system that replaces manual, spreadsheet-based tracking of sales, stock, and installment balances with a single web application. It is not an extension of an existing system and has no legacy data to migrate. The frontend is a React single-page application; the backend is a NestJS service backed by PostgreSQL as the single source of transactional truth, with Redis for caching and Temporal for the durable, time-based logic that installment plans require.

2.2 Product Functions

At a high level, the system performs the following functions. Each is elaborated into detailed requirements, including specific rules and edge cases, in Section 3.1.

Function	Summary
Authentication & RBAC	Secure login and server-enforced permissions for two roles
Catalog & inventory	Product/category management and a ledger-based, auditable stock model
POS checkout	Transaction-safe sales, full-payment or installment
Customer management	Registration, search, and purchase/installment history
Installment management	Schedule calculation and payment recording
Installment automation	Durable reminder and overdue-escalation workflow
Documents	Invoice and receipt generation
Reporting	Sales, inventory, installment, and financial reports

Analytics (optional)	Demand forecasting, risk scoring, reorder suggestions, anomaly flags
----------------------	--

2.3 User Characteristics

Role	Description	Count (typical)
Administrator	Configures products, pricing, and interest rates; reviews sales, payments, and audit records; views dashboards and generates reports. Assumed to have moderate computer literacy and administrative familiarity with the business.	1–2
Worker / Cashier	Operates the POS screen: searches products, registers customers, processes sales and installment payments, prints receipts. Assumed to have basic computer literacy and little to no prior software training.	Several

2.4 Constraints

- Two-person development team with a short, fixed delivery window.
- Fixed technology stack: React, NestJS, PostgreSQL, Redis, Temporal, Docker, GitHub Actions.
- Modular monolith architecture; no microservices, Kubernetes, or Kafka.
- Single retail location (or a small number sharing one database), not a distributed multi-branch chain.

2.5 Assumptions and Dependencies

- Staff have access to a standard, modern web browser and a stable internet connection.
- The business has a single, shared price list and interest-rate policy configured by an Administrator; per-customer negotiated pricing is not assumed.
- Third-party services used in development (e.g., object storage, notification provider) are assumed available in a self-hosted or free-tier form suitable for an academic project.
- No integration with an external accounting or ERP system is assumed for this delivery.

2.6 Requirements Subsets

Every requirement in Section 3.1 is labeled as either core (must be delivered) or optional (delivered only if time remains after the core is complete and tested). This subset boundary is the primary tool

used to manage scope for a two-person team and a short timeline; it is not a suggestion but a firm project decision.

Subset	Requirements included
Core	FR1 Authentication & RBAC, FR2 Product/Inventory, FR3 POS Checkout, FR4 Customer Management, FR5 Installment Sales, FR6 Installment Workflow, FR7 Invoices/Receipts, FR8 Reporting — plus all non-functional requirements in Sections 3.2–3.12 as they apply to the above.
Optional	FR9.1 Demand Forecasting, FR9.2 Installment Risk Scoring, FR9.3 Reorder Recommendation, FR9.4 Anomaly Detection.

3. Specific Requirements

3.1 Functionality

This section describes every functional requirement of the system in detail, organized by feature. FR1–FR8 are core requirements: each is a hard requirement for delivery. FR9 is optional and is broken into four independent sub-features, each of which may be included, partially included, or omitted depending on remaining time, without affecting the core system.

3.1.1 FR1 — User Authentication and Role-Based Access Control

The system shall provide secure login for two roles: Administrator and Worker. This requirement covers the full authentication lifecycle and the enforcement of role-based permissions across every other feature in this document.

- The login screen shall accept an email address and password, and shall display a generic error message on failure without revealing whether the email or the password was incorrect.
- Passwords shall be stored using a strong one-way hash (e.g., bcrypt) and shall never be stored or logged in plain text.
- On successful login, the system shall issue a signed session token (JWT) that encodes the user's role and expires after a configurable period (default: 8 hours), after which the user must log in again.
- Every API endpoint that changes data or exposes sensitive information shall verify the caller's role on the server before executing, regardless of what the client-side interface displays.
- Administrators shall have access to: product and pricing configuration, interest-rate configuration, all reports, audit records, and user management.

- Workers shall have access to: product search, customer registration and search, checkout, installment payment recording, and their own receipt/invoice history.
- A Worker attempting to access an Administrator-only endpoint shall receive an authorization error and the attempt shall be recorded in the audit log.
- The system shall record every login attempt (successful and failed), including timestamp and user email, for audit purposes.
- A logout action shall immediately invalidate the current session token on the client; server-side token revocation before natural expiry is not required for this delivery.

3.1.2 FR2 — Product, Category, and Inventory Management

This requirement covers catalog management and the ledger-based inventory model that keeps stock levels explainable and auditable.

- Administrators shall be able to create, edit, and deactivate product categories (e.g., phones, accessories, laptops).
- Administrators shall be able to create and edit products with, at minimum: name, category, cost price, selling price, and an optional flag indicating whether the product is individually serialized.
- A product shall not be permanently deleted once it has any associated sales history; it may only be deactivated so historical records remain intact.
- Every change to a product's stock quantity shall be recorded as a discrete stock-movement entry with a type, quantity, timestamp, and the user who triggered it, rather than overwriting the stock quantity directly.

Movement type	Effect on stock	Typical trigger
PURCHASE	Increase	New stock received from a supplier
SALE	Decrease	A completed POS checkout
RETURN	Increase	A customer return of previously sold stock
DAMAGED	Decrease	Stock written off as damaged or unsellable
ADJUSTMENT	Increase or decrease	Manual correction after a stock count

- The current stock level for a product shall always be computed as the sum of its stock-movement history, so the figure is fully auditable and re-derivable.

- For products flagged as serialized, the system shall allow an individual unit to be recorded with its own serial number or IMEI, a warranty start date, and a status (in stock, sold, returned).
- Administrators and Workers shall be able to search products by name, category, or serial number, and the search results shall show current stock level.
- The system shall flag a product as low stock when its current stock level falls below a per-product threshold configured by an Administrator, and shall display this flag on the product list.

3.1.3 FR3 — Point-of-Sale (POS) Checkout

This requirement covers the checkout flow used by Workers to record a sale, including the transactional guarantees needed to keep sales, payments, and stock consistent.

- A Worker shall be able to search for and add one or more products to a cart, specifying quantity and an optional per-item discount.
- The system shall calculate a running subtotal, total discount, and grand total as items are added to or removed from the cart.
- At checkout, the Worker shall select a payment type: full payment or installment (deposit).
- For a full-payment checkout, the Worker shall record the payment method (e.g., cash, card) and confirm the sale.
- For an installment checkout, the Worker shall specify a deposit amount before proceeding to FR5 (Installment Sales Management) to define the schedule.

On checkout confirmation, the system shall perform the following sequence as a single atomic database transaction; if any step fails, no part of the sequence is committed:

1. Validate that the customer, products, and prices in the cart still match current catalog data.
 2. Validate that sufficient stock is available for every item in the cart.
 3. Create the sale record and its line items.
 4. Record the payment (full amount, or the deposit for an installment sale).
 5. Create a SALE stock-movement entry for each item and update stock accordingly.
 6. Create an audit-log entry for the completed sale.
 7. Commit the transaction, then generate the invoice (FR7) as a follow-up step outside the transaction boundary.
- The system shall use row-level locking or an equivalent mechanism so that two simultaneous checkouts cannot both sell the same last unit of stock.
 - The system shall attach an idempotency key to each checkout request and shall reject a duplicate submission of the same request rather than creating a second sale.

- Completed sales shall never be deleted; corrections after checkout are handled through a separate compensating record, not a direct edit or deletion of the original sale.

3.1.4 FR4 — Customer Management

This requirement covers customer registration and lookup, which underpins both POS checkout and installment management.

- A Worker shall be able to register a new customer with, at minimum: full name and phone number; email and address are optional.
- The system shall warn the Worker if a new customer's phone number matches an existing customer record, to reduce duplicate entries, but shall not block registration outright.
- A Worker shall be able to search existing customers by name or phone number when starting a sale, and select an existing customer instead of registering a new one.
- Each customer record shall maintain a visible history of past sales and, if applicable, active and completed installment plans, viewable by Administrators and by the Worker currently serving that customer.

3.1.5 FR5 — Installment Sales Management

This requirement covers the creation and ongoing management of installment plans, including schedule calculation and payment recording.

- A Worker shall be able to create an installment plan at checkout by specifying: deposit amount, number of installments, and payment frequency (e.g., monthly).
- The interest rate applied to an installment plan shall come from Administrator-configured settings and shall not be freely editable by a Worker at checkout.
- On plan creation, the system shall calculate a schedule of individual installments, each with an amount and a due date, based on the financed amount (total minus deposit), the interest rate, and the number of installments.
- A Worker shall be able to record a payment against a specific installment, and the system shall update the plan's outstanding balance and mark that installment as paid, partially paid, or unpaid accordingly.
- The system shall correctly handle a partial payment (less than the installment amount), an early payment (before the due date), and an overpayment (applied to the next unpaid installment).
- After each payment is recorded, the system shall recompute the remaining balance and the next due date and reflect them immediately on the customer's installment view.

- An Administrator shall be able to cancel an active installment plan; cancellation shall be recorded as a compensating entry, not a deletion of the plan's history.

3.1.6 FR6 — Installment Reminder and Overdue Workflow

This requirement covers the automated, time-based logic that runs for each active installment plan, implemented as a durable background workflow so it survives restarts and cannot silently stop running.

- For each installment on an active plan, the system shall start a background workflow at plan creation that tracks that installment through to payment or escalation.
- The workflow shall wait until a configurable number of days before the due date (default: 3 days) and then send a reminder notification to the customer.
- On the due date, the workflow shall check whether the installment has been paid; if unpaid, it shall wait through a configurable grace period (default: 5 days) before re-checking.
- If the installment remains unpaid after the grace period, the workflow shall mark the account overdue and surface it on the Administrator dashboard for follow-up.
- If a reminder notification fails to send (e.g., provider error), the workflow shall retry a limited number of times with a backoff delay rather than failing silently.
- The workflow shall resume correctly from its last waiting point after an application or worker-process restart, and shall not send a duplicate reminder for the same installment as a result of a restart.
- If a payment is recorded for an installment while its workflow is waiting, the workflow shall detect the payment on its next check and stop the reminder/escalation sequence for that installment without further action.
- The core delivery shall support at least one notification channel (e.g., email); support for additional channels such as SMS or WhatsApp is an optional enhancement and is not required for the core system to be considered complete.

3.1.7 FR7 — Invoice and Receipt Generation

This requirement covers the generation of downloadable financial documents for completed sales and installment payments.

- On completion of a sale, the system shall generate a PDF invoice with a unique, sequential invoice number, the date and time, customer name, cashier name, line items with quantity and price, any discounts, the total, and the payment method.
- For an installment sale, the invoice shall additionally show the deposit amount, the financed amount, the interest applied, and the full installment schedule.

- Where a sold item is a serialized unit, the invoice shall include its serial number or IMEI and, if applicable, its warranty period.
- Each individual installment payment shall generate a separate PDF receipt referencing the original sale and the specific installment it settles.
- Generated invoices and receipts shall be stored so that an Administrator or the relevant Worker can re-download them later without regenerating the document.

3.1.8 FR8 — Reporting

This requirement covers the operational and financial reports available to Administrators.

Report area	Example metrics	Filters
Sales	Daily/monthly revenue, sales by product or category, sales by cashier	Date range, category, cashier
Inventory	Current stock, stock valuation, low-stock items	Category, stock-level threshold
Installments	Upcoming payments, overdue accounts, outstanding balance by customer	Date range, status (active/overdue/completed)
Financial	Payment summary, daily cash summary, outstanding receivables	Date range, payment method

- An Administrator shall be able to select a report type, apply the relevant filters above, and generate the report as a PDF or CSV export.
- A report covering a large date range shall be generated as a background job so the request does not block the interface; the Administrator shall see a pending status and receive a download link once the file is ready.
- Previously generated reports shall remain available for re-download for a reasonable retention period without being regenerated.

3.1.9 FR9 — Analytics and Decision Support (Optional)

The following four sub-features are optional. They are attempted only after FR1–FR8 are implemented and tested, may be delivered partially or not at all depending on remaining time, and shall not alter core transactional behavior in any way. Each is listed as an independent sub-feature so it can be included or dropped individually.

3.1.9.1 FR9.1 — Demand Forecasting

- The system may estimate expected product demand over a short horizon (e.g., 7 or 14 days) using historical sales data for a product or category.
- The forecast shall be presented to Administrators as a decision-support figure only; it shall never automatically alter stock levels, pricing, or ordering.
- Where implemented, the chosen forecasting method shall be evaluated against a simple baseline (e.g., a moving average) using a standard error metric before being presented as the system's recommendation.

3.1.9.2 FR9.2 — Installment Payment-Risk Scoring

- The system may classify each active installment account as low, medium, or high risk, based on factors such as prior late payments, missed payments, and remaining balance.
- The risk score shall be used only to help Administrators prioritize follow-up; it shall never automatically cancel a plan, deny a future installment sale, or take any other automated action against a customer.

3.1.9.3 FR9.3 — Reorder Recommendation

- The system may suggest a reorder quantity for a product, combining forecast demand, current stock, and a configured safety-stock margin.
- The recommendation shall remain advisory only; creating an actual purchase order is outside the scope of this delivery, and an Administrator makes any resulting procurement decision manually.

3.1.9.4 FR9.4 — Anomaly Detection

- The system may flag unusually large discounts, unusual sales volumes, or unusual refund patterns using a simple statistical rule (e.g., values well outside a normal range for that product or cashier).
- A flagged transaction shall be surfaced to an Administrator for review; it shall never be blocked or reversed automatically.

3.2 Usability

3.2.1 Learnability

A Worker with basic computer literacy shall be able to complete a full-payment sale within 10 minutes of first using the system, without formal training beyond a short walkthrough of the checkout screen.

3.2.2 Efficiency for Repeat Use

A Worker who has used the system for at least one week shall be able to complete a routine full-payment checkout for a single item in under 30 seconds of active screen interaction, excluding customer-facing conversation time.

3.2.3 Error Prevention and Recovery

The system shall validate required fields before allowing a form to be submitted and shall display an inline, specific error message next to the offending field rather than a generic failure message. Actions that cannot be undone (e.g., cancelling an installment plan) shall require an explicit confirmation step.

3.2.4 Interface Consistency

Navigation, button placement, and terminology shall be consistent across all screens, so that a pattern learned on one screen (e.g., how search works, how a confirm action is triggered) applies on every other screen.

3.3 Reliability

3.3.1 Transactional Integrity

No sale, payment, or inventory update shall be left in a partially applied state; the atomic-transaction behavior specified in FR3 shall hold under all tested failure conditions, including a database error or a server crash mid-checkout.

3.3.2 Availability

The core system shall be available during all demonstration, testing, and normal usage periods; brief downtime for deployment of a new build is acceptable, provided it does not coincide with a scheduled demonstration.

3.3.3 Data Durability

A recorded financial transaction (sale, payment, or installment entry) shall never be lost due to an application or worker-process restart; in-progress installment workflows shall resume from where they left off rather than restarting or being silently skipped, per FR6.

3.3.4 Accuracy

All monetary calculations (totals, discounts, installment schedules, interest) shall be accurate to two decimal places and shall use a consistent rounding rule applied uniformly across the system, to avoid cumulative rounding discrepancies in reports.

3.3.5 Defect Severity Classification

Defects found during testing shall be classified as critical (data loss, incorrect financial totals, or an unusable core flow), major (a core flow works but produces a wrong non-financial result or a poor

experience), or minor (cosmetic or edge-case issues). Critical defects shall be resolved before any core requirement is considered complete.

3.4 Performance and Security

3.4.1 Response Time

Operation	Target (average)	Target (maximum)
Login	< 500 ms	< 1.5 s
Product search	< 500 ms	< 1.5 s
Checkout confirmation (≤ 10 items)	< 1 s	< 2 s
Report generation (small range)	< 2 s	< 5 s

3.4.2 Throughput and Capacity

The system shall support at least 5 concurrent checkout operations without a measurable increase in response time beyond the targets above, which comfortably covers the expected usage of a single retail location with a small number of simultaneous Workers.

3.4.3 Resource Utilization and Degradation

Under normal load, the application server's memory and CPU usage shall remain stable rather than growing over time (indicating a leak). If a supporting service becomes unavailable, the system shall degrade gracefully rather than failing completely:

- If Redis is unavailable, the system shall fall back to querying PostgreSQL directly for data normally served from cache, at the cost of slower response times, rather than failing the request.
- If the Temporal service is unavailable, existing installment workflows shall resume once it recovers rather than losing their state; POS checkout, which does not depend on Temporal, shall continue to function normally in the meantime.
- If report generation fails, the report job shall be marked as failed with a visible status rather than left in a permanently pending state.

3.4.4 Security Controls

- Passwords shall be stored using a strong one-way hash and shall never be logged or transmitted in plain text.
- All traffic between client and server shall be encrypted in transit using HTTPS.

- Access to Administrator-only functionality shall be enforced on the server, independent of what the client interface displays or hides.
- All user-supplied input shall be validated and sanitized on the server before use in a database query, to prevent injection attacks.
- Secrets (database credentials, signing keys) shall be stored outside source control, using environment configuration rather than being hard-coded.
- API endpoints shall apply basic rate limiting to reduce the impact of abusive or automated request patterns.
- Every change to price, stock, a payment, or a user role shall be recorded in an audit log with the acting user, timestamp, and a description of the change.

3.5 Supportability

3.5.1 Coding Standards

The codebase shall follow a consistent, linted TypeScript style across frontend and backend, with automatic formatting applied before code is merged.

3.5.2 Modularity

The backend shall be organized into clearly bounded modules by business domain (authentication, catalog, sales, installments, reporting, and so on), so a change in one area has a limited, predictable effect on others.

3.5.3 Configuration Management

Environment-specific values (database connection, secrets, feature flags) shall be supplied through environment configuration, not hard-coded, so the same build can run in development and in a deployed environment without code changes.

3.5.4 Documentation

A README shall describe environment setup, how to run the system locally, and how to run the automated test suite, so a new contributor can get the system running without direct help from the original developers.

3.5.5 Logging

The backend shall emit structured logs (JSON) for key operational events — checkout completion, payment failure, and installment-workflow retries or failures — using the framework's built-in logger.

This is sufficient to satisfy the project's operational-awareness requirement without a dedicated metrics or dashboard stack; see Section 3.6.3 for the corresponding excluded technologies.

3.6 Design Constraints

3.6.1 Technology Stack Constraint

The system shall be built with React (frontend), NestJS (backend), PostgreSQL (database), Redis (cache), and Temporal (installment workflows), per the approved project architecture. Substituting a different database, backend framework, or workflow engine is out of scope without a documented change to this SRS.

3.6.2 Architectural Constraint

The backend shall be built as a modular monolith rather than a microservices architecture, to avoid distributed-systems overhead that is not justified by the project's scale or timeline.

3.6.3 Explicitly Excluded Technologies

Technology	Reason for exclusion
LLM / vector database	Deterministic reporting and conventional ML solve the core decision-support needs more predictably.
Kafka	No event throughput in this project justifies a distributed streaming platform.
Microservices	A modular monolith gives simpler transactional coordination at this scale.
Kubernetes / Helm	Not required to demonstrate a working, production-oriented system for this delivery.
Data lakehouse	PostgreSQL reporting views are sufficient at this data volume.
Deep learning	Conventional forecasting and tabular methods suit the available data better.
Reverse proxy (Nginx/Caddy)	Only needed for a self-managed VM with a custom domain; unnecessary when deploying behind a platform that manages HTTPS termination automatically.

Full observability stack (Prometheus/Grafana)	Structured application logging (Section 3.5.5) is sufficient to demonstrate operational awareness at this project's scale; the full metrics/dashboard stack is an optional later extension.
---	---

3.6.4 Team and Timeline Constraint

The system shall be delivered by a two-person team within the project's allotted timeframe. Scope beyond FR1–FR8 is treated as optional and shall not be attempted at the expense of core-requirement quality or completeness.

3.7 On-line User Documentation and Help System Requirements

Beyond inline field labels, placeholder text, and validation messages, no dedicated in-app help system (such as a searchable knowledge base or guided tour) is required for this delivery. A short, separate user guide covering setup and basic day-to-day operation shall be provided alongside the software.

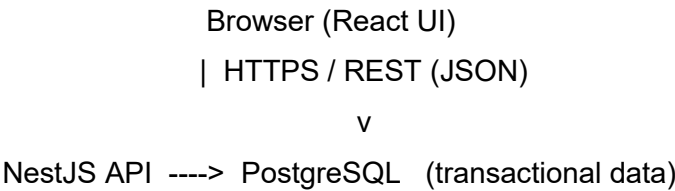
3.8 Purchased Components

No commercially licensed components are used in this system. The following third-party software is open-source and used under its respective license, with no purchase or paid subscription required for development or for the delivered system.

Component	Role
NestJS, Prisma	Backend framework and database ORM
React	Frontend framework
PostgreSQL, Redis	Database and cache
Temporal	Durable workflow orchestration for installment reminders
Docker, GitHub Actions	Containerization and CI/CD

3.9 Interfaces

The diagram below shows the main interfaces between the components a user interacts with and the services behind them.



```
| ----> Redis      (cache, rate limit)
| ----> Temporal   (installment workflows)
| ----> Object store (generated PDFs)
```

3.9.1 User Interfaces

- Login page: email and password fields, an error-message area for invalid credentials.
- POS / checkout screen: product search box, cart list with quantity and per-item discount, customer lookup, deposit field (installment only), payment method selection, and a confirm-sale button.
- Product & inventory screen: product list/table with stock and low-stock indicator, an add/edit form, and a stock-movement history view per product.
- Installments screen: plan-creation form (deposit, term count, frequency), a schedule table showing each installment's amount, due date, and status, and a record-payment action per installment.
- Customer screen: registration form, search box, and a customer detail view showing purchase and installment history.
- Reports screen: report-type dropdown, filter controls matching the table in FR8, and PDF/CSV export buttons.
- Admin dashboard: summary cards for today's sales, outstanding installment balance, low-stock items, and overdue installments.
- All screens are reached through a left-hand navigation menu scoped to the logged-in user's role.

3.9.2 Hardware Interfaces

The client requires only a device with a modern web browser, at least 4 GB of RAM, and a stable internet connection; no specialized client hardware is required. A receipt printer, if used, is accessed through the browser's standard print function rather than a dedicated driver.

3.9.3 Software Interfaces

The frontend communicates with the backend over HTTPS using REST/JSON. The backend integrates with PostgreSQL (via an ORM) for transactional data, Redis for caching and rate limiting, Temporal's SDK for workflow orchestration, S3-compatible object storage for generated PDFs, and GitHub Actions for the CI/CD pipeline.

3.9.4 Communications Interfaces

All client-server communication uses HTTPS. Internal communication between the backend and supporting services (Temporal, object storage) occurs over a private Docker network in development and a private network in deployment.

3.10 Database Requirements

The system requires a relational schema in PostgreSQL. The core entities are listed below; referential integrity shall be enforced with foreign keys, schema changes shall be managed through versioned migrations, and financial and stock-movement records shall never be destructively deleted — corrections are made through compensating entries instead.

Entity	Purpose
Users	Administrator and Worker accounts, including hashed passwords and role.
Customers	Customer contact details and links to sales and installment history.
Products, Categories	Catalog data, pricing, and category grouping.
StockMovements	The full, immutable ledger of stock changes per product.
SerializedUnits	Individual serial-number/IMEI-tracked product instances.
Sales, SaleItems	Completed transactions and their line items.
Payments	Payments recorded against a sale or an installment.
InstallmentPlans, InstallmentSchedules	Installment agreements and their per-installment schedule.
Invoices	Generated invoice and receipt metadata and storage references.
Reports	Metadata for generated report jobs and their download location.
AuditLogs	A record of sensitive actions across the system, per Section 3.4.3.

3.11 Licensing, Legal, Copyright, and Other Notices

This system is developed for academic purposes as a university project and is not intended for commercial distribution. Customer data collected (name, contact details, purchase history) shall be accessible only to authorized roles and used only for the operation of the system, and shall not be shared with any third party outside the scope of running the application.

3.12 Applicable Standards

This SRS follows the general structure of IEEE 830-1998. The system's API follows standard REST conventions and HTTP status codes; dates are represented in ISO 8601 format; and general security practices follow the OWASP Top 10 guidance as a baseline reference for the controls in Section 3.4.3.

3.13 Detailed Scenario Walkthroughs

This section walks through four representative scenarios end to end, tracing exactly what the system does at each step. These walkthroughs are provided in addition to the itemized requirements above to remove any ambiguity about sequencing and system behavior for the most important flows.

3.13.1 Scenario: Full-Payment Checkout

8. A Worker searches for a product by name and adds it to the cart, entering a quantity of 1.
9. The Worker searches for the customer by phone number and selects the matching record; if no record exists, the Worker registers a new customer inline.
10. The system displays the running subtotal and total; the Worker applies no discount and selects “cash” as the payment method.
11. The Worker confirms the sale. The system validates stock availability, opens a database transaction, creates the sale and its line item, records the payment, creates a SALE stock movement, writes an audit-log entry, and commits.
12. The system generates a PDF invoice and displays a confirmation screen with a link to download or print it.
13. The product's stock level, now reduced by one unit, is immediately reflected on the inventory screen.

3.13.2 Scenario: Installment Checkout and First Missed Payment

14. A Worker adds a product to the cart and selects “installment” as the payment type.
15. The Worker enters a deposit amount and a term of 3 monthly installments. The system applies the Administrator-configured interest rate and calculates a schedule of 3 installments with amounts and due dates.
16. The Worker confirms the sale; the deposit is recorded as a payment, stock is deducted, and an invoice showing the full schedule is generated, per FR3 and FR5.
17. A background workflow starts for each of the 3 installments, per FR6.
18. Three days before the first installment's due date, the workflow sends a reminder to the customer.

19. On the due date, no payment has been recorded. The workflow waits through the 5-day grace period.
20. The grace period elapses with still no payment. The workflow marks the account overdue, and it appears on the Administrator dashboard's overdue-installments list.
21. The customer later visits and pays the overdue installment. A Worker records the payment; the workflow detects it on its next check and stops the escalation sequence for that installment without further action.

3.13.3 Scenario: Monthly Sales Report

22. An Administrator opens the Reports screen, selects “Sales” as the report type, and sets the date range to the previous calendar month.
23. The Administrator selects PDF as the export format and submits the request.
24. Because the date range is large, the system queues the request as a background job and shows a pending status rather than blocking the screen.
25. A worker process generates the report from the transactional sales data, writes the PDF to object storage, and marks the job complete.
26. The Administrator sees the job status change to complete and downloads the finished report from the same screen.

3.13.4 Scenario: Concurrent Checkout on the Last Unit of Stock

27. A product has exactly one unit remaining in stock. Two Workers, at two different terminals, both add it to a cart at nearly the same time.
28. Worker A confirms checkout first. The system acquires a lock on that product's stock row, validates that one unit is available, and proceeds with the transaction described in FR3.
29. Worker B confirms checkout moments later, while Worker A's transaction is still committing. The system's locking mechanism causes Worker B's request to see the now-updated stock level of zero.
30. Worker B's checkout fails stock validation and the system displays a clear “out of stock” message instead of completing a sale against a product with no remaining inventory.

3.13.5 Scenario: Stock Correction After a Physical Count

31. An Administrator performs a physical stock count and finds that a product's actual count is two units lower than the system's recorded stock level.
32. The Administrator opens the product's stock-movement history and creates a manual ADJUSTMENT movement of -2, entering a short reason (e.g., “physical count correction”).

33. The system records the adjustment with the Administrator's user ID and a timestamp, and the product's computed stock level immediately reflects the correction.
34. The original discrepancy remains visible in the movement history rather than being erased, so a later audit can see both the correction and the reason for it.

4. Supporting Information

4.1 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE, 1998.
- NestJS Documentation, <https://docs.nestjs.com> (Accessed 15 Aug 2026).
- Temporal Documentation, <https://docs.temporal.io> (Accessed 15 Aug 2026).
- Prisma Documentation, <https://www.prisma.io/docs> (Accessed 15 Aug 2026).
- PostgreSQL Documentation, <https://www.postgresql.org/docs> (Accessed 15 Aug 2026).
- OWASP Top 10, <https://owasp.org/www-project-top-ten> (Accessed 15 Aug 2026).

4.2 Appendix A — Role Permission Summary

Function	Administrator	Worker
Product & pricing configuration	Yes	No
Interest-rate configuration	Yes	No
POS checkout	Yes	Yes
Customer registration & search	Yes	Yes
Installment payment recording	Yes	Yes
Installment plan cancellation	Yes	No
Reports & dashboards	Yes	No
Audit log access	Yes	No

4.3 Appendix B — Core vs Optional Requirements Summary

Requirement	Status
FR1–FR8 and Sections 3.2–3.12 as applied to them	Core — required for delivery
FR9.1 Demand Forecasting	Optional
FR9.2 Installment Risk Scoring	Optional
FR9.3 Reorder Recommendation	Optional
FR9.4 Anomaly Detection	Optional

4.4 Appendix C — Data Dictionary

This appendix expands each entity introduced in Section 3.10 into its individual fields, to remove ambiguity about exactly what data the system stores.

Users

Field	Type	Description
id	UUID	Primary key

email	String	Unique login identifier
password_hash	String	One-way hash of the password; never stored in plain text
role	Enum	ADMINISTRATOR or WORKER
created_at	Timestamp	Account creation time

Customers

Field	Type	Description
id	UUID	Primary key
full_name	String	Required
phone	String	Required; used for duplicate-warning search
email	String	Optional
address	String	Optional

Products & Categories

Field	Type	Description
id	UUID	Primary key
name	String	Required
category_id	UUID	Foreign key to Categories
cost_price / selling_price	Decimal	Two-decimal precision, per Section 3.3.4
is_serialized	Boolean	Whether individual units are tracked by serial number
low_stock_threshold	Integer	Per-product reorder-flag threshold
active	Boolean	False once deactivated; never physically deleted once sold

StockMovements

Field	Type	Description
id	UUID	Primary key
product_id	UUID	Foreign key to Products
type	Enum	PURCHASE, SALE, RETURN, DAMAGED, or ADJUSTMENT — see Section 3.1.2
quantity	Integer	Positive or negative, per movement type
created_by	UUID	Foreign key to Users
created_at	Timestamp	Movement time

Sales, SaleItems & Payments

Field	Type	Description
sale.id	UUID	Primary key
sale.customer_id	UUID	Foreign key to Customers
sale.cashier_id	UUID	Foreign key to Users (Worker who processed it)
sale_item.product_id, quantity, unit_price, discount	Mixed	One row per line item
payment.amount, method, sale_id	Mixed	One row per payment; multiple rows possible for an installment plan

InstallmentPlans & InstallmentSchedules

Field	Type	Description
plan.id	UUID	Primary key
plan.sale_id	UUID	Foreign key to Sales
plan.deposit, interest_rate, term_count	Mixed	Set at plan creation, per Section 3.1.5
schedule.plan_id	UUID	Foreign key to InstallmentPlans
schedule.due_date, amount, status	Mixed	One row per installment; status is PENDING, PAID, or OVERDUE

SerializedUnits

Field	Type	Description
id	UUID	Primary key
product_id	UUID	Foreign key to Products
serial_number	String	Unique serial number or IMEI
warranty_start_date	Date	Used to compute warranty expiry
status	Enum	IN_STOCK, SOLD, or RETURNED

Invoices

Field	Type	Description
id	UUID	Primary key
sale_id	UUID	Foreign key to Sales
invoice_number	String	Unique, sequential
document_url	String	Location of the stored PDF

type	Enum	INVOICE or RECEIPT, per Section 3.1.7
------	------	---------------------------------------

Reports & AuditLogs

Field	Type	Description
report.id, type, filters, status	Mixed	Tracks a report job from QUEUED to COMPLETED, per FR8
report.document_url	String	Location of the generated file once complete
audit_log.actor_id	UUID	Foreign key to Users — who performed the action
audit_log.action, entity, entity_id	Mixed	What was changed and on which record
audit_log.created_at	Timestamp	When the action occurred

4.5 Appendix D — Key Validation Rules

Field / action	Rule
Customer phone number	Required; system warns (does not block) on a likely duplicate match
Product selling price	Must be a positive number with at most two decimal places
Checkout with empty cart	Blocked; confirm-sale action is disabled until at least one item is added
Checkout stock validation	Blocked if any item's quantity exceeds current available stock at confirmation time
Installment deposit	Must be less than the sale total, so a financed balance greater than zero remains
Installment term count	Must be a whole number between 1 and an Administrator-configured maximum
Login email/password	Both required; generic error on failure, per Section 3.1.1
Report date range	End date must not be before start date; both required before the export button is enabled

4.6 Appendix E — Testing Strategy Summary

This appendix summarizes, at a level consistent with an SRS, the categories of testing that verify the requirements above. Detailed test cases are maintained separately from this document.

Test category	What it verifies
Unit tests	Installment schedule calculation, payment allocation, discount and total calculation, stock-movement arithmetic — FR2, FR3, FR5
Integration tests	Database transaction behavior and Temporal workflow activities — FR3, FR6

End-to-end tests	Full checkout, installment creation, invoice generation, and report generation flows — FR3, FR5, FR7, FR8
Concurrency tests	Last-unit-in-stock checkout and duplicate checkout submission, per Section 3.13.4 — FR3
Failure tests	Notification-provider failure and retry, and workflow resumption after a restart — FR6

4.7 Appendix F — API Endpoint Overview

This appendix lists the main REST endpoints implied by the requirements above, for reference during design. Exact routes and payloads are finalized during implementation; this is a requirements-level overview, not an API specification.

Method	Endpoint	Purpose
POST	/auth/login	Authenticate a user and issue a session token — FR1
GET	/products	Search and list products — FR2
POST	/products	Create a product (Administrator only) — FR2
POST	/products/:id/stock-movements	Record a stock movement — FR2
POST	/customers	Register a customer — FR4
GET	/customers?search=	Search customers — FR4
POST	/sales	Perform a checkout (full payment or installment deposit) — FR3, FR5
POST	/installments/:id/payments	Record a payment against an installment — FR5
GET	/installments/overdue	List overdue installment accounts — FR6
GET	/sales/:id/invoice	Download a sale's invoice PDF — FR7
POST	/reports	Queue a report generation job — FR8
GET	/reports/:id	Check report job status and retrieve the download link — FR8

4.8 Appendix G — Non-Functional Requirements Summary

This appendix consolidates the non-functional requirements from Sections 3.2–3.6 into a single reference table.

Category	Requirement	Target / rule
Usability	Learnability	First sale completed within 10 minutes, no formal training
Usability	Efficiency	Routine single-item checkout under 30 seconds after one week of use

Reliability	Transactional integrity	No partially applied sale, payment, or stock update
Reliability	Accuracy	Monetary values accurate to two decimal places, consistent rounding
Performance	Checkout response time	Average < 1 s, maximum < 2 s for up to 10 items
Performance	Concurrency	5 concurrent checkouts without target degradation
Security	Data protection	Hashed passwords, HTTPS in transit, server-enforced RBAC
Supportability	Documentation	README sufficient for independent local setup
Design constraint	Technology stack	React, NestJS, PostgreSQL, Redis, Temporal, Docker only
Design constraint	Architecture	Modular monolith; no microservices, Kubernetes, or Kafka
Design constraint	Reverse proxy / observability	Both optional; core uses platform-managed HTTPS and structured logging

No further appendices are included at this stage. The SmartRetail project proposal document should be read alongside this SRS for architectural rationale and the phased delivery roadmap.